

NHN NAN 2026 · 게임 × AI 해커톤 사전과제

최후의 벽, 최후의 사람

게임 소개서 — 개요 · 플레이 방법 · 실행 방법

개인 참가 · ry-eon

플레이 <https://ry-eon.github.io/NAN2026-NHN-Game-X-AI-Hackathon/>

소스 · 커밋 기록 <https://github.com/ry-eon/NAN2026-NHN-Game-X-AI-Hackathon>

2026-08-10

한 줄 소개

성벽 위 포문이 향하는 방향이 곧 적이 오는 방향이다. 화망을 그려 언데드 군세의 흐름을 직접 조종하는 3D 농성전.

이 게임의 한 가지 아이디어

타워 디펜스는 보통 "적이 오는 길목에 무엇을 놓을까"를 묻는다. 이 게임은 반대로 묻는다 — "적을 어디로 오게 만들까."

괴수는 성벽 위 화력이 두꺼운 구간을 피해 **빈 곳으로 흐른다**. 그래서 플레이어는 대포를 한쪽으로 몰아 겨누고, 반대편을 **일부러 비운다**. 비워둔 그곳이 킬존이다.

성립하는 이유는 괴수가 보는 것이 제한돼 있기 때문이다: **괴수는 성벽 위만 본다**. 성문 안쪽 지상에 선 영웅은 그들의 계산에 들어가지 않는다. 무방비로 보이는 구간에 몰려든 무리를, 기다리던 영웅이 광역 스킬로 쓸어담는다.

게임 방법

1) 준비 — 병사를 병기에 붙인다

플레이어는 **성주** 한 사람이다. 그런데 판이 시작될 때 **대포와 발리스타는 비어 있다**.

수비병 12명이 안뜰에 모여 있고, 이들을 병기 곁으로 보내 **장착**해야 비로소 포가 불을 뿜는다. 빼면 그 병기는 즉시 침묵하고, 다시 붙이면 재개한다. 미장착 병기는 바닥에 흰 링이 깜빡인다. **아무것도 하지 않으면 성은 반드시 무너진다** — 배치가 이 게임의 첫 수다.

B를 누르면 전원이 알아서 자기 자리를 찾아간다. 배치가 끝나면 **Space**로 침공을 시작한다.

2) 침공 — 화망을 그린다

동쪽 정면에서 주력이 밀려오고, 일부는 **모서리를 돌아** 북·남벽의 동쪽 끝을 친다.

대포와 발리스타는 **고정 병기다. 옮길 수 없다**. 대신 드래그로 선택해 **우클릭으로 겨눈다**. 겨눈 방향이 곧 화망이고, 화망이 괴수의 진로를 바꾼다. 조준선이 주황색으로 표시된다.

한 가지 규칙이 리듬을 만든다: **괴수의 목표는 몰려오는 시점에 정해지고 이후 바뀌지 않는다**. 늦게 돌린 포문은 지금 오는 무리에게는 통하지 않고 **다음 웨이브부터** 통한다. 그래서 판단은 웨이브와 웨이브 사이에 내려진다. 화면 상단에 **웨이브 진행과 다음 웨이브까지의 시간**이 표시된다.

3) 세 사람의 스킬 — 각자 다른 일을 한다

영웅 둘과 성주가 각각 **Q · W · E** 세 개를 쓴다. **E**는 궁극기다.

	Q	W	E (궁극)
전사 (근접)	돌진 — 경로의 적을 베며 파고든다	회전베기	대지파쇄 — 광역 피해 + 기절
마법사 (화염)	화염구	불의 장막 — 10초간 타는 장판	업화 — 대형 광역
성주 (지휘)	군기 — 주변 병기 재장전 2배	진군 나팔 — 이동 속도	총력전 — 전군 강화

전사는 칼이 닿는 거리에서만 싸운다. 마법사는 성벽 위에 세우는 편이 낫다 — 안뜰에 두면 사거리 대부분을 성벽에 버린다. 성주의 버프는 **성주를 따라다니는 오라라**, 버프를 들고 전선으로 걸어가는 플레이가 생긴다.

4) 백병전 — 농성을 접는다

V 를 누르면 수비병 전원이 병기를 버리고 **성문 밖으로 나가** 싸운다. 대가는 분명하다 — 그동안 성벽 화력은 **전부 멈춘다**. 다시 살리려면 B 로 불러들여야 하고, 그 왕복 시간이 곧 이 결단의 비용이다.

5) 클라이맥스 — 네크로맨서

막판에 **이 군세를 일으킨 자가** 정문으로 걸어온다. 성문 앞에 버티고 서서 **쓰러진 것들을 다시 세운다**. 잡몹을 붙잡고 있으면 판이 끝나지 않는다 — 자동 사격은 가까운 적부터 치므로 술사를 영영 건드리지 않는다. **포문을 돌려 술사를 먼저 끊을지**가 이 판의 마지막 판단이다.

승패

- **패배**: 성벽 HP가 0이 된다
- **승리**: 모든 웨이브를 격퇴한다
- 1라운드 고정. 한 판은 약 2분.
- 판 종료 후 R 또는 종료 카드 버튼으로 **같은 시드로 재시작**(결정론 유지)

조작

조작	동작
좌클릭 드래그	부대 선택 (Shift 추가 · Ctrl 클릭/더블클릭 = 같은 병종 전체)
우클릭	병기 = 조준 / 병사·영웅·성주 = 이동 / 병기를 직접 클릭 = 장착하러 간다
Q W E	선택한 영웅·성주의 스킬 (지점 스킬은 좌클릭으로 시전, 우클릭 취소)
B / V	자동 장착 / 총 백병전(성문 밖 출격)
A / S · H	어택땅 / 홀드 — 홀드가 아니면 근접 병사는 알아서 붙어 싸운다
F1 / F2	영웅 순환 / 전군 선택
Ctrl + 1 ~ 9 / 1 ~ 9	부대 지정 / 호출
마우스 휠 · 화살표 · 화면 가장자리	줌 · 카메라 이동 (Space / C 성주 시점 복귀)
Space · ESC · R · M	침공 개시 · 선택 해제 · 재시작 · 음소거

실행 방법

웹에서 바로 (권장)

링크 클릭만으로 플레이됩니다. 설치·로그인 없음.

- **플레이**: <https://ry-eon.github.io/NAN2026-NHN-Game-X-AI-Hackathon/>
- **권장 환경**: 데스크톱 Chrome / Edge (WebGL2). 내장 GPU(Intel UHD 630)를 기준으로 만들었고, 프레임이 모자라면 해상도를 자동으로 낮춘다(0.7~1.5배).

로컬에서

```
pnpm install
pnpm dev      # http://localhost:5173
```

링크

항목	주소
플레이 (GitHub Pages)	https://ry-eon.github.io/NAN2026-NHN-Game-X-AI-Hackathon/
소스 · 커밋 기록	https://github.com/ry-eon/NAN2026-NHN-Game-X-AI-Hackathon
플레이 영상 (30~60초)	(촬영 후 기재)

왜 게임 안에서 LLM을 실시간으로 부르지 않았나

게임×AI 해커톤이니 "플레이 중에 AI가 말을 걸거나 콘텐츠를 즉석에서 만드는" 형태를 먼저 떠올리게 된다. 우리는 그 자리를 **의도적으로 비웠다**. AI는 게임 안이 아니라 **게임을 만드는 파이프라인에 있다**. 이유는 셋이다.

1) 운영 비용 — 재미가 호출 수에 비례해 돈이 된다

런타임에 LLM을 부르면 **플레이 1회마다 비용이 발생한다**. 사람이 많이 즐길수록 적자가 커지는 구조라, 충분한 인프라와 단가 협상이 전제된 조직에서만 성립한다. 사전과제 요건이 "런타임에 유료 API·라이선스 금지"인 것도 같은 현실을 반영한 것으로 읽었다. 우리는 **AI 산출물을 빌드 타임에 코드로 굳혀** 런타임 비용을 0으로 만들었다.

2) 보안 — 상호작용을 유지하려면 유저 정보가 밖으로 나간다

대화형 상호작용은 **컨텍스트 유지**가 전제다. 그러려면 유저의 입력과 플레이 상태를 외부 상용 API로 계속 보내야 하고, 그 순간 **유저 정보 취급·보관·위탁** 문제가 따라온다. 게임 하나를 재미있게 만들자고 감당할 비용이 아니라고 봤다. 지금 이 게임은 **외부로 나가는 데이터가 없다**. 정적 파일만으로 돌아간다.

3) 보증 불가 — 실시간 생성은 "게임으로 성립함"을 증명할 수 없다

이게 가장 결정적이었다. LLM이 판을 실시간으로 만들면 **매 판이 다르다**. 같은 조건에서 같은 결과가 나오지 않으니 재현이 안 되고, 재현이 안 되면 검증도 못 한다. 그러면 우리가 내세우는 문장 — **"만든 것이 게임으로 성립함을 시스템이 증명한다"** — 이 성립하지 않는다. **결정론을 포기하는 순간 보증도 함께 사라진다**.

그래서 AI를 어디에 뒀나

처음 기획할 때부터 관심은 **콘텐츠 생성**과 **실제 운영** 둘 다였다. 둘을 동시에 만족시키는 자리가 파이프라인이었다.

[생성] AI가 규칙·콘텐츠·밸런스 후보를 만든다

↓

[검증] 봇 4등급이 판을 직접 플레이해 판정한다 (반려되면 되돌아간다)

↓

[출고] 통과한 것만 정적 빌드로 굳는다 — 런타임에 AI 없음

이 구조 덕분에 **빠르게 만들고 빠르게 버릴 수 있었다**. 밸런스 후보 13종, 편성 4종, 스킬 사거리 조합 18종을 봇으로 돌려 숫자로 골랐다. 사람이 손으로 플레이해 골랐다면 불가능한 양이다. 실패한 안들은 그대로 기록에 남아 「AI 활용 기술 문서」의 증거가 됐다.

정리하면 — AI를 런타임에서 빼는 것이 이 프로젝트에서는 기술적 후퇴가 아니라 선택이었다. 비용 0, 외부 전송 0, 매 판 재현 가능, 그리고 검증 가능한 게임이 그 대가로 남았다.

기술 요약 (상세는 「AI 활용 기술 문서」)

- **Three.js + TypeScript + Vite.** 런타임에 유료 API·라이선스 없음. 정적 파일만으로 동작한다.
 - **결정론 시뮬레이션:** 게임 규칙은 렌더링을 모르는 순수 TypeScript sim에 있고, 같은 (시드, 입력)이면 항상 같은 결과가 나온다.
 - **봇이 검증하고 나서야 출고한다:** 봇 4등급(무개입 / 표준 장착 / 적극 플레이 / 무작위 조작)이 플레이어와 같은 입력 타입으로 판을 돌려 판정한다. 현재 시드 6종 **전부 통과**. 판정 기준에는 “아무것도 안 하면 져야 한다”가 들어 있다 — 배치가 게임에 의미 있음을 기계가 확인하는 항목이다.
 - 외부 에셋은 전부 CC0(Poly Haven, ambientCG). 캐릭터·괴수·보스 모델은 전량 코드 생성이고, **효과음 18종도 전부 WebAudio 합성**이다(오디오 파일 0개).
-